Lab 3: Path Traversal

Senal Goonetilaka

136471174

Seneca Polytechnic

WAS600 - Web Security

February 03, 2026

Table of Contents

Table of Figures

Lab Report: Path Traversal

Introduction

This lab explores **path traversal vulnerabilities** through six challenges on the PortSwigger Web Security Academy platform. Path traversal, also known as directory traversal, is a common web security flaw that allows attackers to access files outside of the intended directory by manipulating file path parameters in application requests. The challenges cover a range of scenarios, from basic relative path traversal to more advanced techniques such as absolute path bypasses, bypassing non-recursive filters with nested sequences, double URL-encoding, prepending expected base paths, and using null byte injection to bypass file extension checks.

Each challenge was completed using **Burp Suite** to intercept and modify HTTP requests targeting the application's image loading functionality. The goal across all six labs was to retrieve the contents of the /etc/passwd file from the server, demonstrating how improper handling of user input in file path operations can lead to unauthorized access to sensitive system files.

Challenge 1 - File path traversal, simple case

Vulnerability Overview

This lab demonstrates a basic path traversal vulnerability in the display of product images. The application loads images using a filename parameter without implementing any validation or sanitization on the user-supplied input, allowing an attacker to manipulate the parameter with directory traversal sequences to access arbitrary files on the server's filesystem.

Objective

The objective of this lab is to exploit the path traversal vulnerability to retrieve the contents of the /etc/passwd file from the server.

Methodology and Exploitation Steps

1. The lab environment was accessed, and the image loading mechanism was identified. The filename parameter in the image request URL was modified directly in the browser to include directory traversal sequences pointing to /etc/passwd.

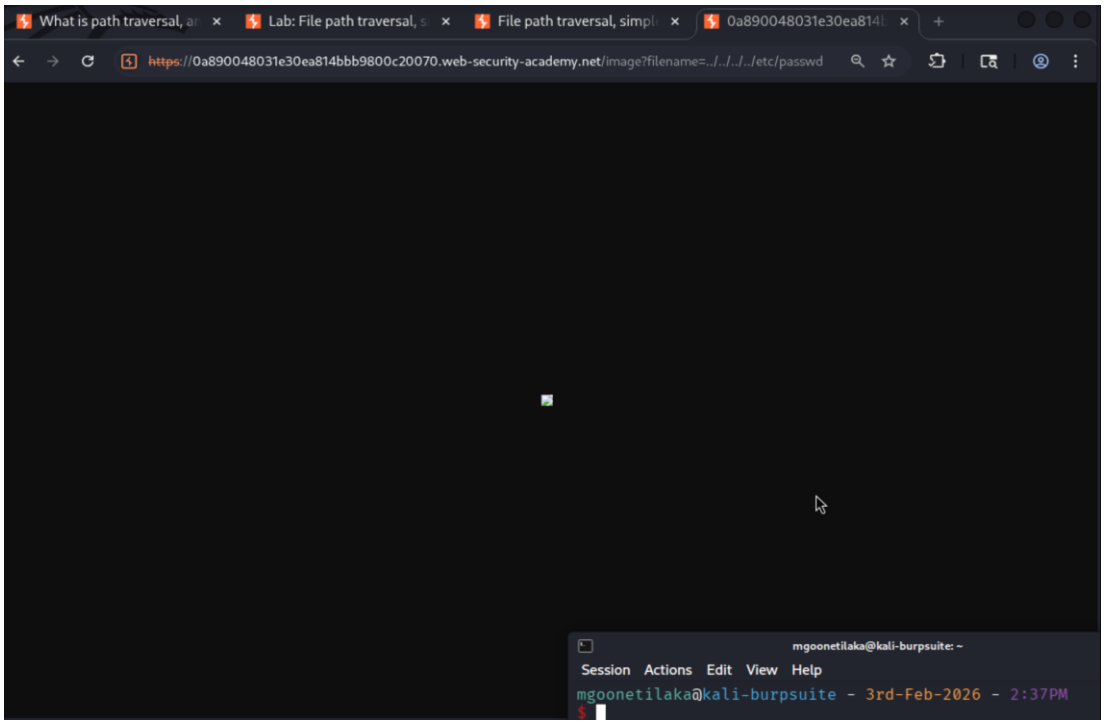

Figure 1: Path traversal payload entered in browser URL

This screenshot **(Figure 1)** shows the browser URL modified to include the traversal payload ../../../../etc/passwd in the filename parameter. The page displayed a broken image icon, as the browser attempted to render the file contents as an image.

2. The request was intercepted and sent to Burp Suite Repeater for further analysis. The filename parameter was modified with traversal sequences targeting **/etc/passwd**, and the request was sent.

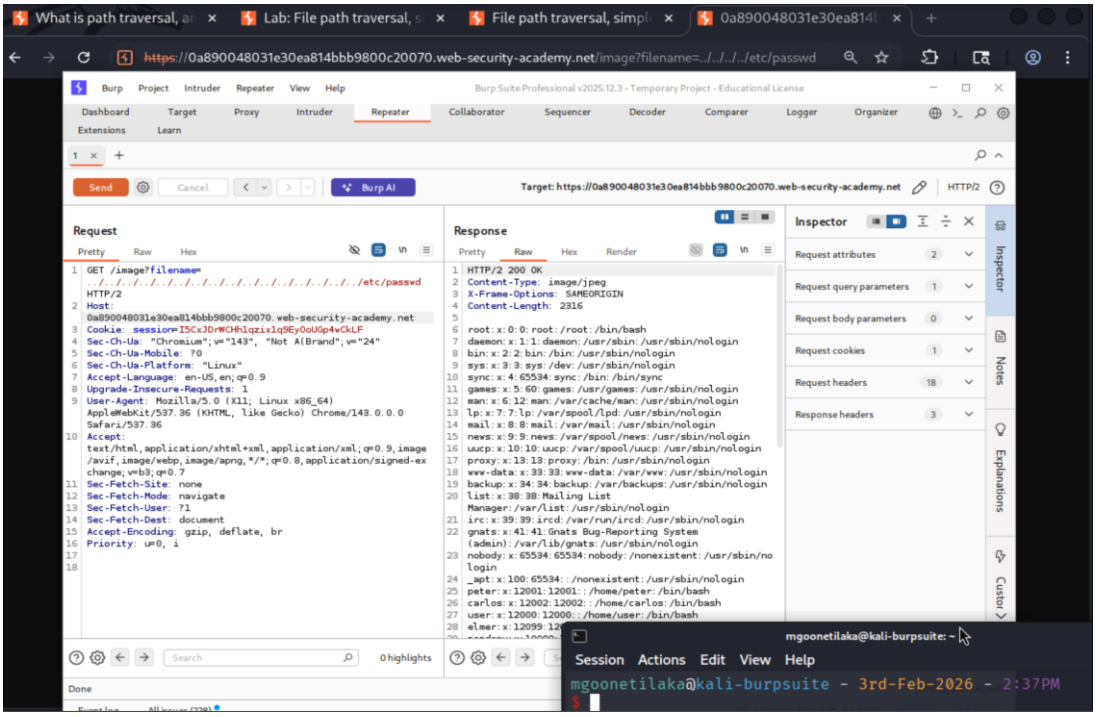

Figure 2: Burp Repeater showing /etc/passwd file contents in response

This screenshot **(Figure 2)** shows the modified GET request in Burp Repeater with the traversal payload in the filename parameter. The server responded with HTTP/2 200 OK and returned the full contents of the /etc/passwd file, confirming the path traversal vulnerability.

3. The lab was marked as solved after successfully retrieving the /etc/passwd file contents.

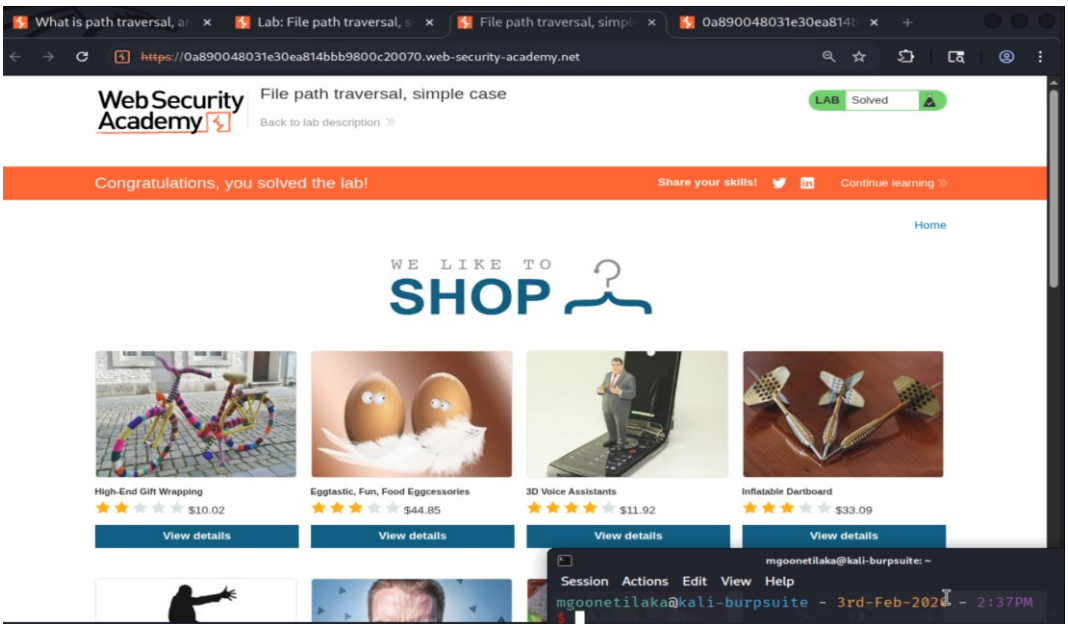

Figure 3: Lab solved confirmation

This screenshot **(Figure 3)** shows the application displaying the "Congratulations, you solved the lab!" confirmation message, indicating successful exploitation of the path traversal vulnerability.

Challenge 2 - File path traversal, traversal sequences blocked with absolute path bypass

Vulnerability Overview

This lab demonstrates a **path traversal** vulnerability where the application blocks relative directory traversal sequences (such as ../) but still treats the supplied filename as being relative to a default working directory. Because the application does not restrict the use of **absolute file paths**, an attacker can bypass the filtering by specifying the full path to a target file directly.

Objective

The objective of this lab is to bypass the traversal sequence filtering and retrieve the contents of the /etc/passwd file from the server using an absolute path.

Methodology and Exploitation Steps

1. The lab environment was accessed, and the filename parameter in the image request URL was modified directly in the browser to use an absolute path (/etc/passwd) instead of relative traversal sequences.

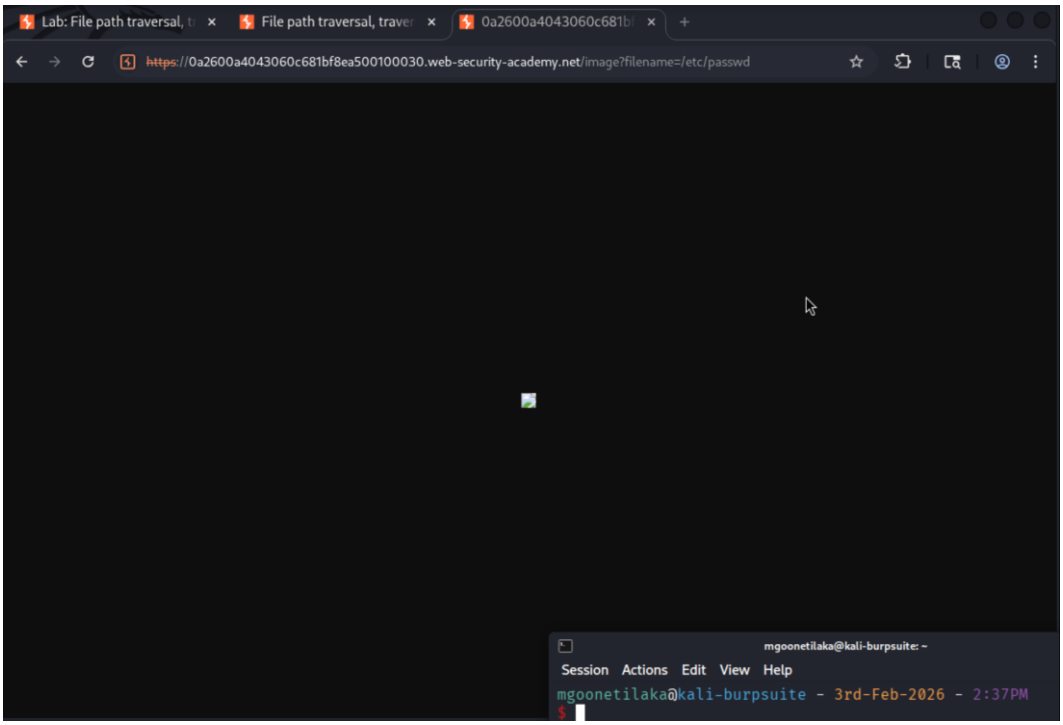

Figure 4: Absolute path payload entered in browser URL

This screenshot **(Figure 4)** shows the browser URL modified to include the absolute path /etc/passwd in the filename parameter. The page displayed a broken image

icon, as the browser attempted to render the returned file contents as an image, confirming that the server processed the request.

2. The request was intercepted and sent to Burp Suite Repeater for detailed analysis. The filename parameter was set to /etc/passwd and the request was sent.

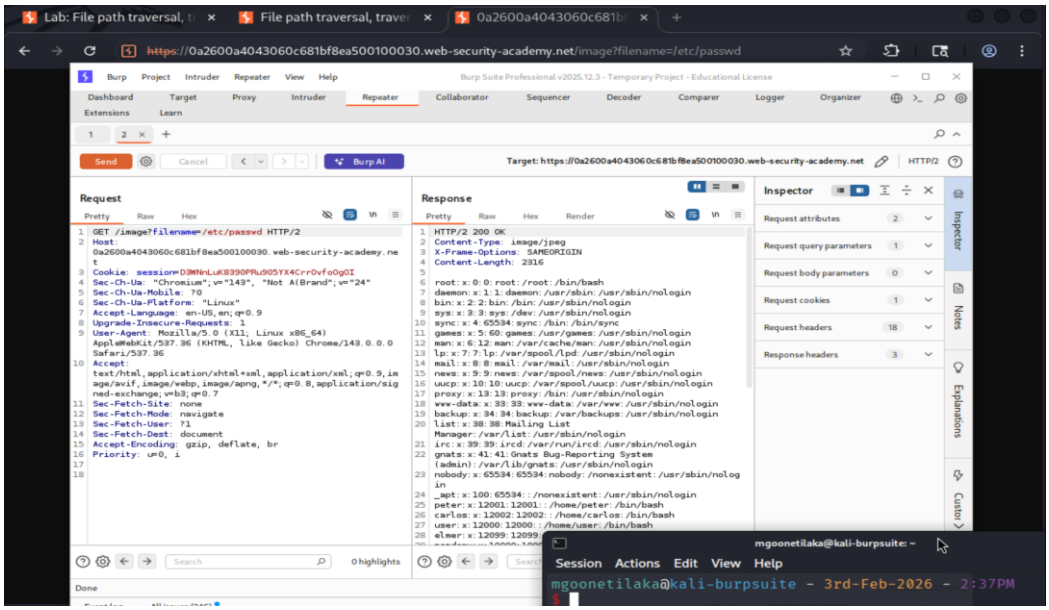

Figure 5: Burp Repeater showing /etc/passwd file contents in response

This screenshot **(Figure 5)** shows the modified GET request in Burp Repeater with the filename parameter set to /etc/passwd as an absolute path. The server responded with HTTP/2 200 OK and returned the full contents of the /etc/passwd file in the response body, confirming that the traversal sequence filter could be bypassed using an absolute path.

3. The lab was marked as solved after successfully retrieving the /etc/passwd file contents.

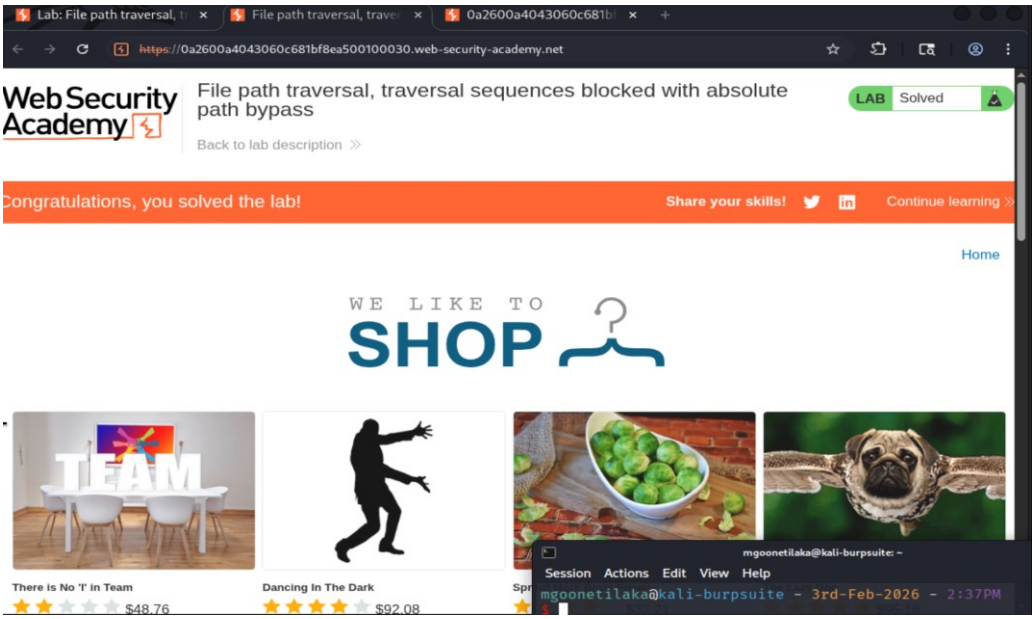

Figure 6: Lab solved confirmation

This screenshot **(Figure 6)** shows the application displaying the "Congratulations, you solved the lab!" confirmation message with the lab status marked as "Solved", indicating successful exploitation of the path traversal vulnerability using the absolute path bypass technique.

Challenge 3 - File path traversal, traversal sequences stripped non-recursively

Vulnerability Overview

This lab demonstrates a **path traversal** vulnerability where the application applies a filter to strip traversal sequences (../) from user-supplied input. However, the stripping is performed **non-recursively**, meaning it only removes the first occurrence of the pattern in a single pass. By nesting traversal sequences within each other, an attacker can craft a payload that, after the filter removes the inner ../, still produces valid traversal sequences that the application processes.

Objective

The objective of this lab is to bypass the non-recursive traversal sequence filter and retrieve the contents of the /etc/passwd file from the server.

Methodology and Exploitation Steps

1. The lab environment was accessed, and the filename parameter in the image request URL was modified directly in the browser using nested traversal sequences. The payload ....//....//....//....//etc/passwd was used, which after the non-recursive stripping of ../ would collapse into ../../../../etc/passwd.

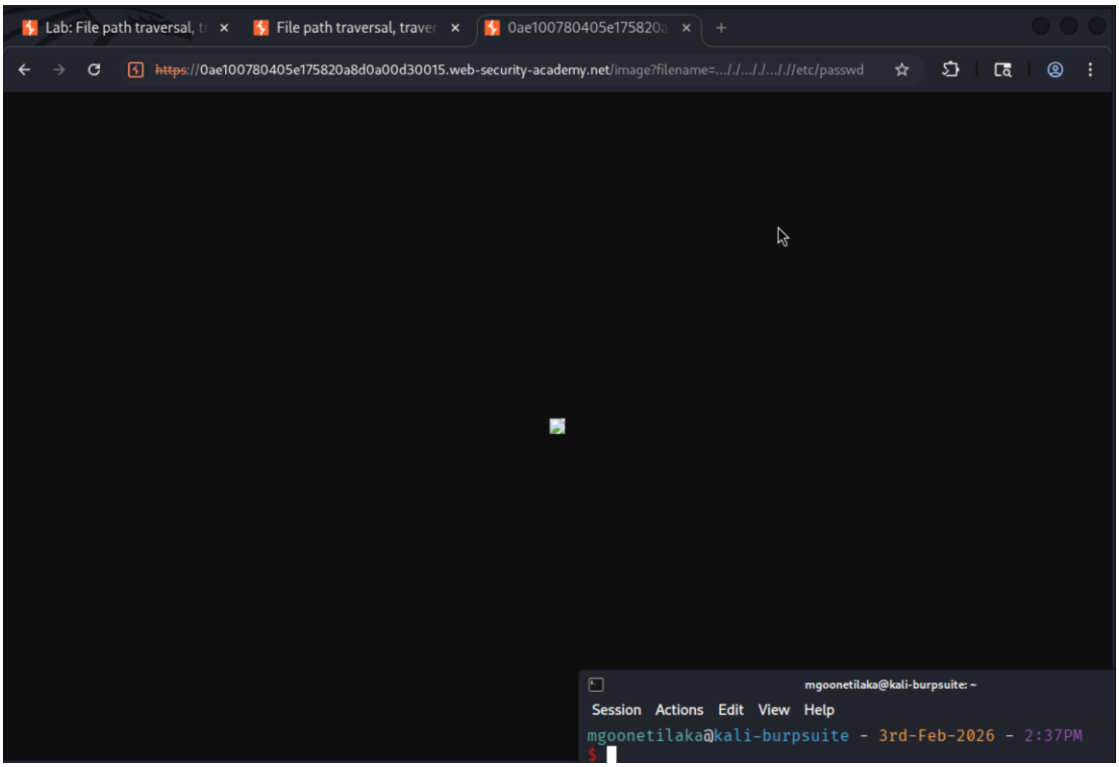

Figure 7: Nested traversal payload entered in browser URL

This screenshot **(Figure 7)** shows the browser URL modified to include the nested traversal payload ....//....//....//....//etc/passwd in the filename parameter. The page displayed a broken image icon, indicating that the server returned non-image content and confirming that the file was being read.

2. The request was captured in Burp Suite Proxy and reviewed in the HTTP history tab. The filename parameter contained the nested traversal payload ....//....//....//etc/passwd, and the server responded with HTTP/2 200 OK, returning the full contents of the /etc/passwd file.

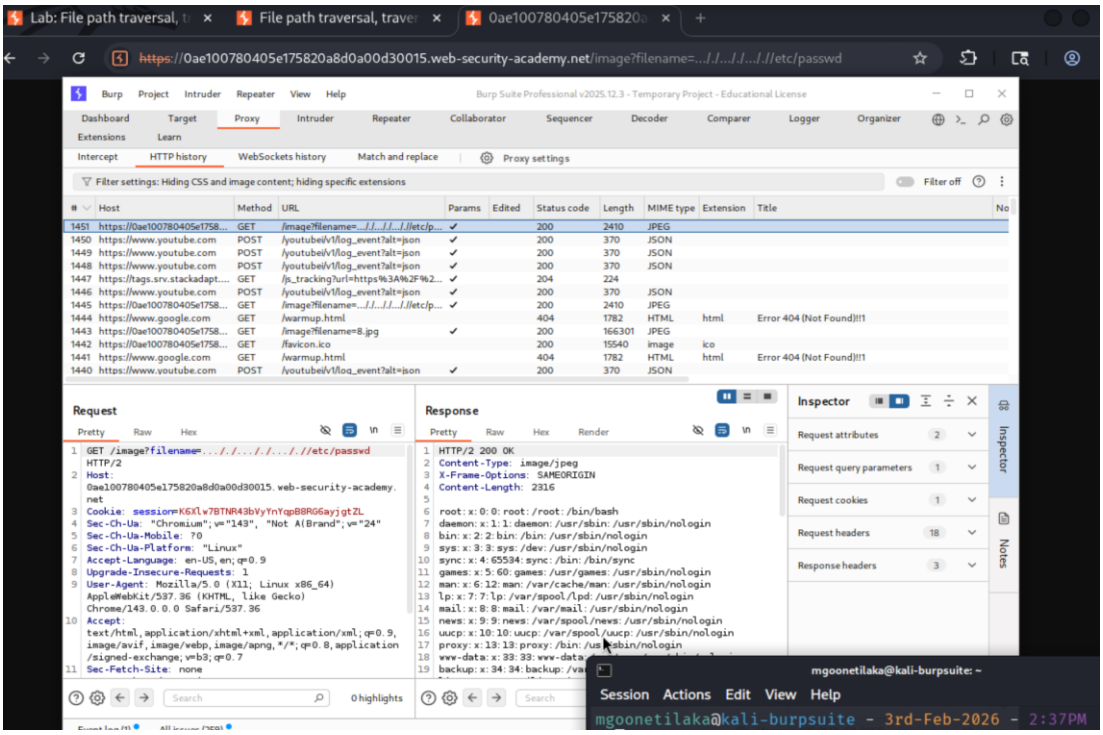

Figure 8: Burp Proxy HTTP history showing successful traversal with ....// payload

This screenshot **(Figure 8)** shows the Burp Suite Proxy HTTP history with the intercepted request highlighted. The request pane displays the GET request with the nested ....// traversal sequences in the filename parameter, and the response pane confirms a 200 OK status with the /etc/passwd file contents returned in the response body.

3. An alternative bypass method was also tested using the ..././ nested sequence. The payload ..././..././..././..././etc/passwd was used, which similarly collapses into valid traversal sequences after the non-recursive stripping removes the inner ../ patterns.

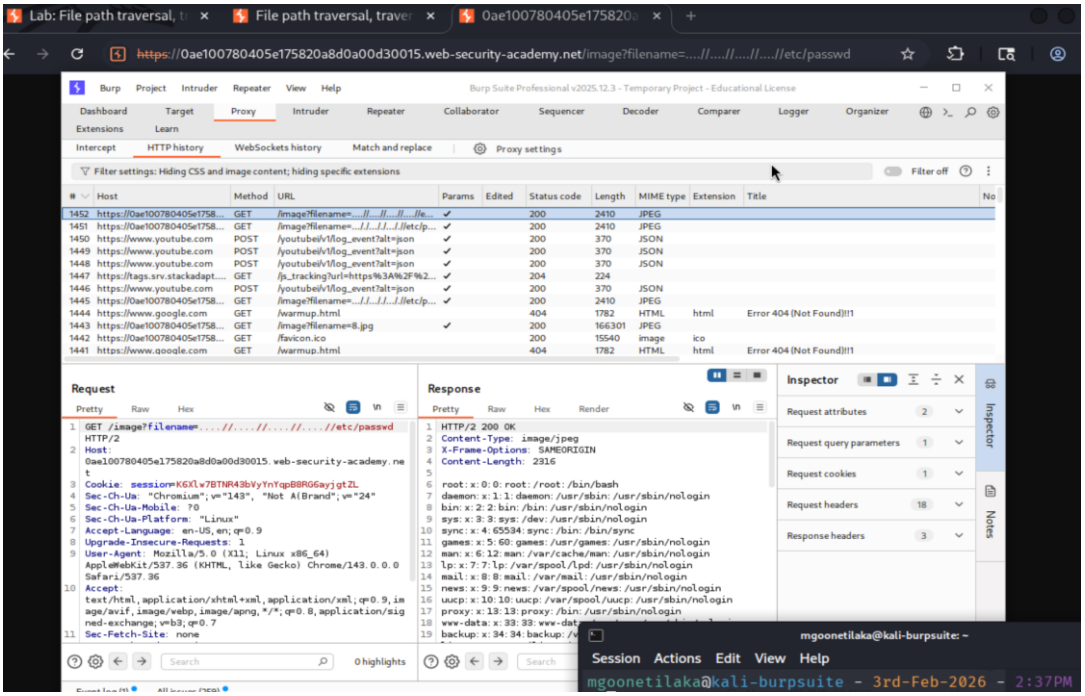

Figure 9: Alternative ..././ nested traversal payload in Burp Proxy

This screenshot **(Figure 9)** shows the Burp Suite Proxy HTTP history with the alternative nested traversal payload ..././..././..././..././etc/passwd in the filename parameter. The server again responded with HTTP/2 200 OK and returned the /etc/passwd file contents, confirming that both nested payload variations successfully bypass the non-recursive filter.

4. The lab was marked as solved after successfully retrieving the /etc/passwd file contents using the nested traversal payload.

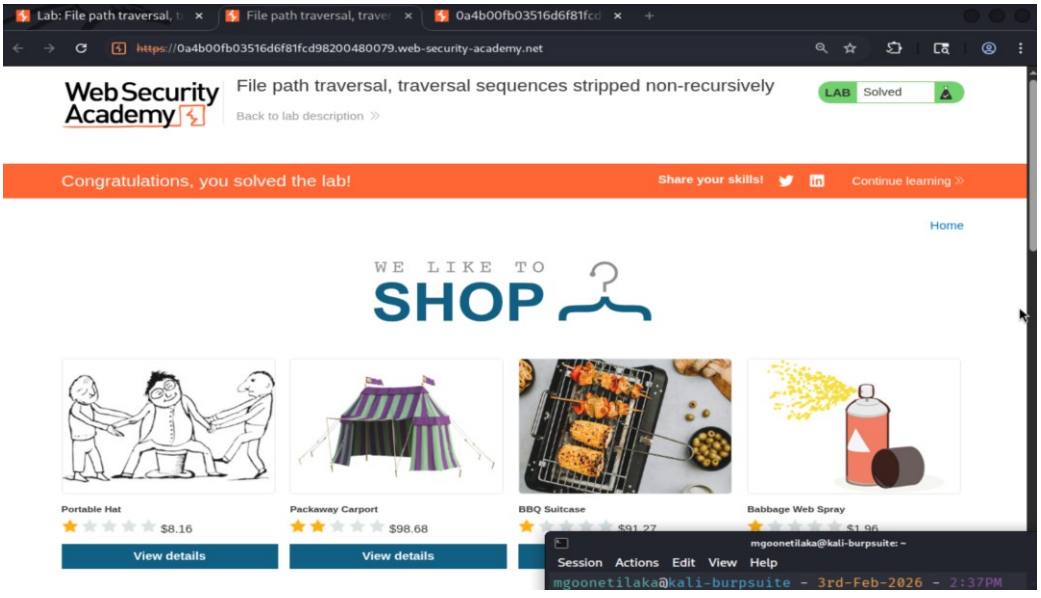

Figure 10: Lab solved confirmation

This screenshot **(Figure 10)** shows the application displaying the "Congratulations, you solved the lab!" confirmation message with the lab status marked as "Solved", indicating successful exploitation of the path traversal vulnerability by bypassing the non-recursive stripping filter.

Challenge 4 - File path traversal, traversal sequences stripped with superfluous URL-decode

Vulnerability Overview

This lab demonstrates a **path traversal** vulnerability where the application strips standard traversal sequences (../) before processing the input. However, the application performs **URL decoding after the validation step**, meaning that URL-encoded or double URL-encoded traversal sequences can bypass the filter. By double-encoding the ../ characters, the payload passes through the initial sanitization intact and is then decoded by the server into valid traversal sequences.

Objective

The objective of this lab is to bypass the URL-decode-based traversal filter and retrieve the contents of the /etc/passwd file from the server using double URL-encoded traversal sequences.

Methodology and Exploitation Steps

1. The lab environment was accessed, and the filename parameter was first tested with a standard traversal payload (../../../../etc/passwd). The server returned a "No such file" error, confirming that the application was stripping or blocking standard traversal sequences.

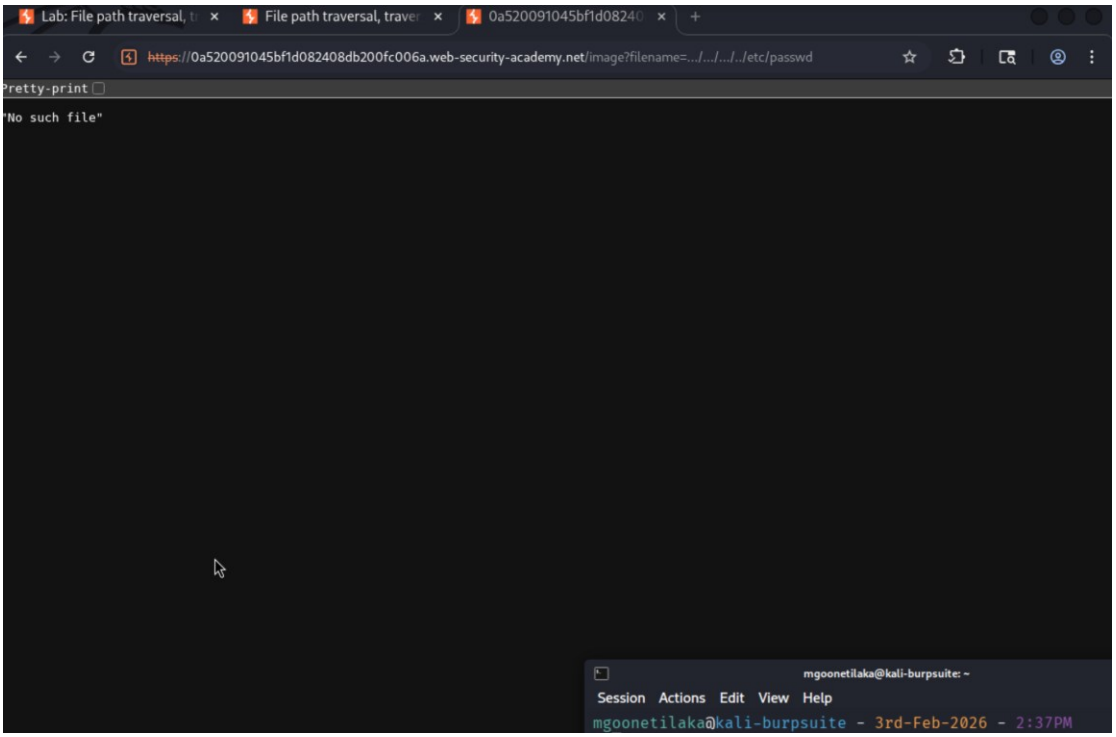

Figure 11: Standard traversal payload blocked by the application

This screenshot **(Figure 11)** shows the browser returning a "No such file" response when a standard ../../../../etc/passwd traversal payload was used in the filename parameter, indicating that the application is filtering traversal sequences before processing the file request.

2. The request was sent to Burp Suite Repeater, and the traversal sequences were **double URL-encoded**. Each . was encoded as %25%32%65 and each / was encoded as %25%32%66 (the double-encoded representations), causing the initial filter to fail to recognize the traversal pattern. After the filter passed the input, the server performed an additional URL decode, which resolved the sequences into valid ../ traversal characters and returned the contents of /etc/passwd.

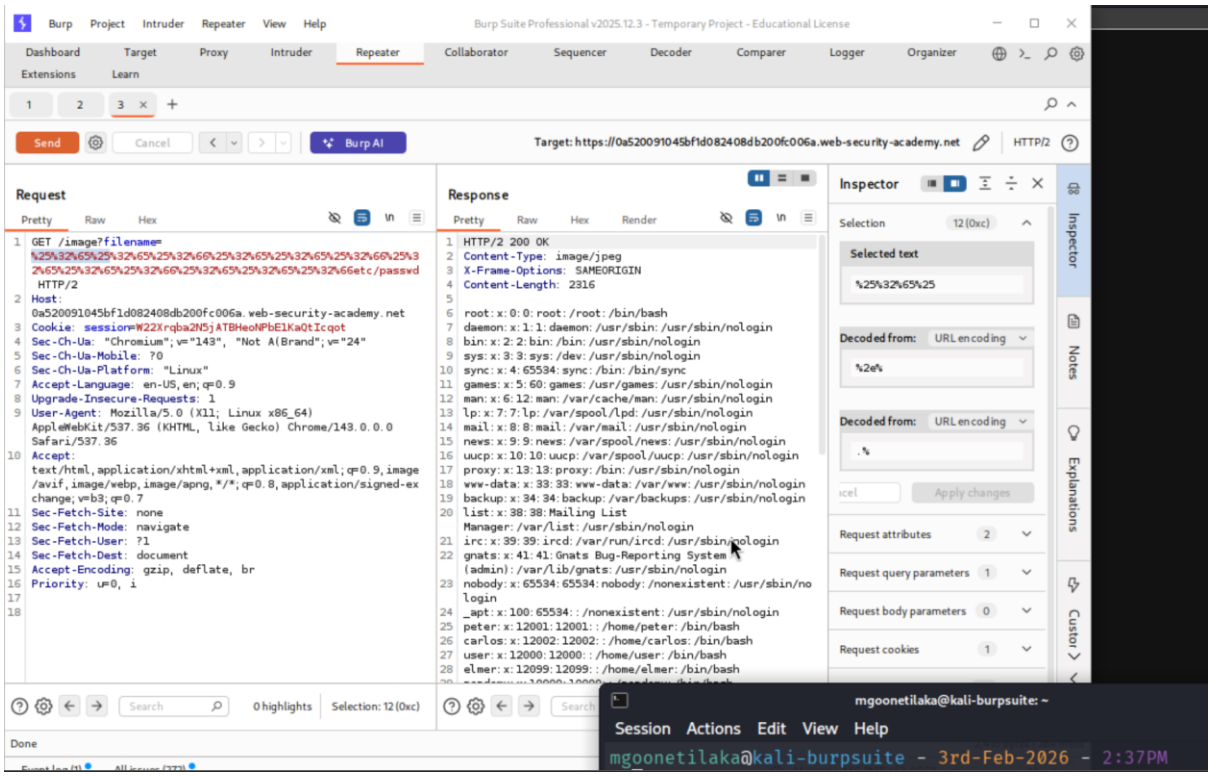

Figure 12: Double URL-encoded traversal payload in Burp Repeater

This screenshot **(Figure 12)** shows the Burp Suite Repeater with the double URL-encoded traversal payload in the filename parameter. The Inspector panel on the right confirms the double-encoding by showing the selected text decoding from %25%32%65%25 through two rounds of URL decoding to the . character. The server responded with HTTP/2 200 OK and returned the full contents of the /etc/passwd file, confirming the successful bypass of the traversal filter.

3. The lab was marked as solved after successfully retrieving the /etc/passwd file contents using the double URL-encoded traversal payload.

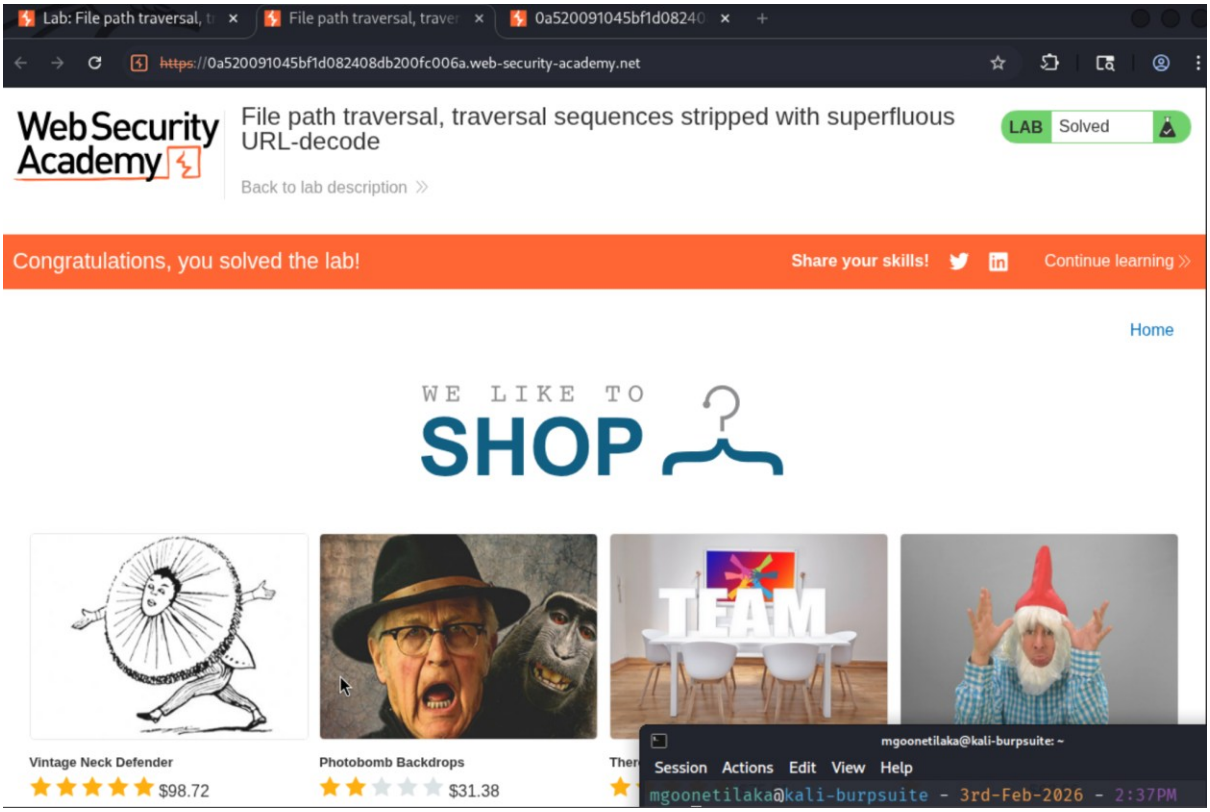

Figure 13: Lab solved confirmation

This screenshot **(Figure 13)** shows the application displaying the "Congratulations, you solved the lab!" confirmation message with the lab status marked as "Solved", indicating successful exploitation of the path traversal vulnerability using the double URL-encoding bypass technique.

Challenge 5 - File path traversal, validation of start of path

Vulnerability Overview

This lab demonstrates a **path traversal** vulnerability where the application validates that the user-supplied filename begins with an expected base folder path (/var/www/images). However, the application does not properly prevent traversal sequences that are appended after the required base path. By including the expected base folder prefix followed by ../ traversal sequences, an attacker can satisfy the validation check while still navigating to arbitrary files on the server's filesystem.

Objective

The objective of this lab is to bypass the start-of-path validation and retrieve the contents of the /etc/passwd file by prepending the required base folder to the traversal payload.

Methodology and Exploitation Steps

1. The lab environment was accessed, and the filename parameter in the image request URL was modified directly in the browser. The payload /var/www/images/../../../etc/passwd was used, which satisfies the validation check by starting with the expected base path while using traversal sequences to navigate to the target file.

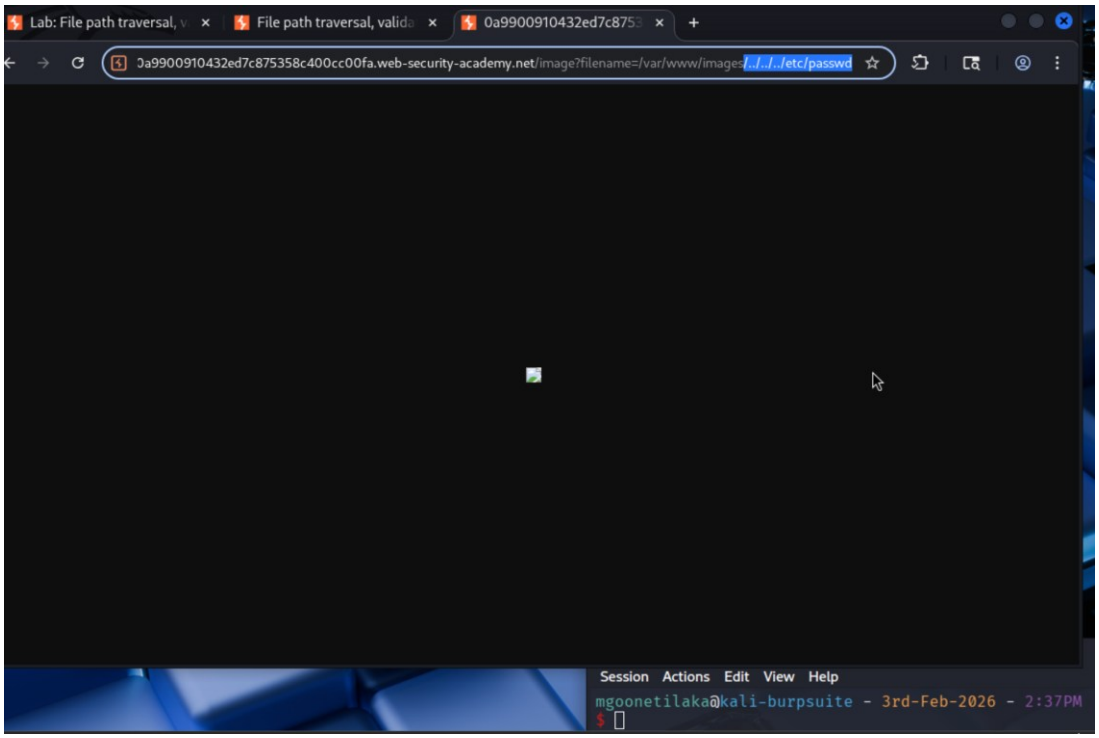

Figure 14: Base path with traversal payload entered in browser URL

This screenshot **(Figure 14)** shows the browser URL modified to include the payload /var/www/images/../../../etc/passwd in the filename parameter. The page displayed a broken image icon, indicating that the server returned non-image content and confirming that the file was being read.

2. The request was sent to Burp Suite Repeater for detailed analysis. The filename parameter was set to /var/www/images/../../../etc/passwd and the request was sent.

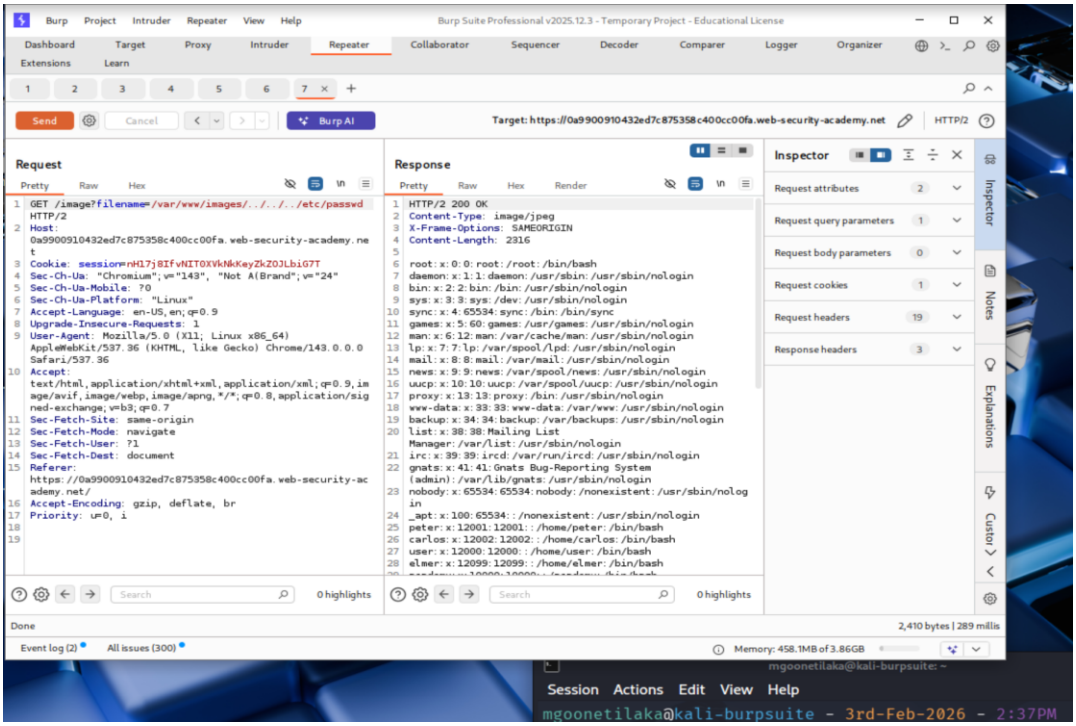

Figure 15: Burp Repeater showing /etc/passwd file contents in response

This screenshot **(Figure 15)** shows the modified GET request in Burp Suite Repeater with the filename parameter set to /var/www/images/../../../etc/passwd. The server responded with HTTP/2 200 OK and returned the full contents of the /etc/passwd file in the response body, confirming that the start-of-path validation was successfully bypassed by including the required base folder prefix before the traversal sequences.

3. The lab was marked as solved after successfully retrieving the /etc/passwd file contents.

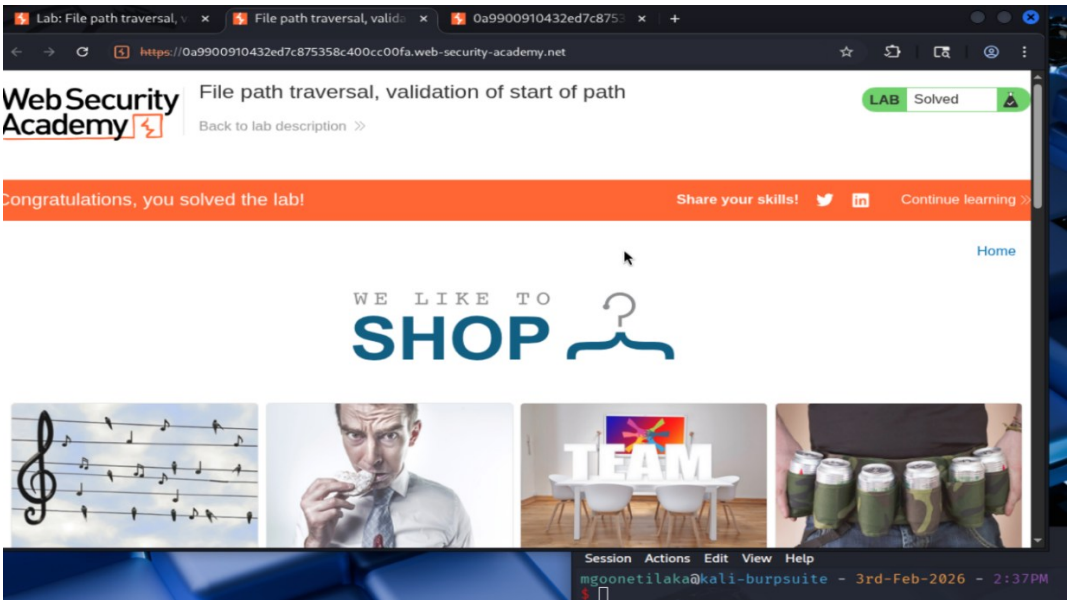

Figure 16: Lab solved confirmation

This screenshot **(Figure 16)** shows the application displaying the "Congratulations, you solved the lab!" confirmation message with the lab status marked as "Solved", indicating successful exploitation of the path traversal vulnerability using the start-of-path validation bypass technique.

Challenge 6 - File path traversal, validation of file extension with null byte bypass

Vulnerability Overview

This lab demonstrates a **path traversal** vulnerability where the application validates that the user-supplied filename ends with an expected file extension, such as .jpg. However, the application is vulnerable to a **null byte injection** attack. By inserting a URL-encoded null byte (%00) before the required file extension, the application's validation check sees the .jpg extension and allows the request, but the underlying filesystem API terminates the string at the null byte and ignores everything after it, effectively reading the file specified before the null character.

Objective

The objective of this lab is to bypass the file extension validation and retrieve the contents of the /etc/passwd file from the server using a null byte injection technique.

Methodology and Exploitation Steps

1. The lab environment was accessed, and the filename parameter was first tested with a standard traversal payload (../../../../etc/passwd). The server returned a "No such file" error, confirming that the application was enforcing file extension validation and rejecting requests that did not end with an expected image extension.

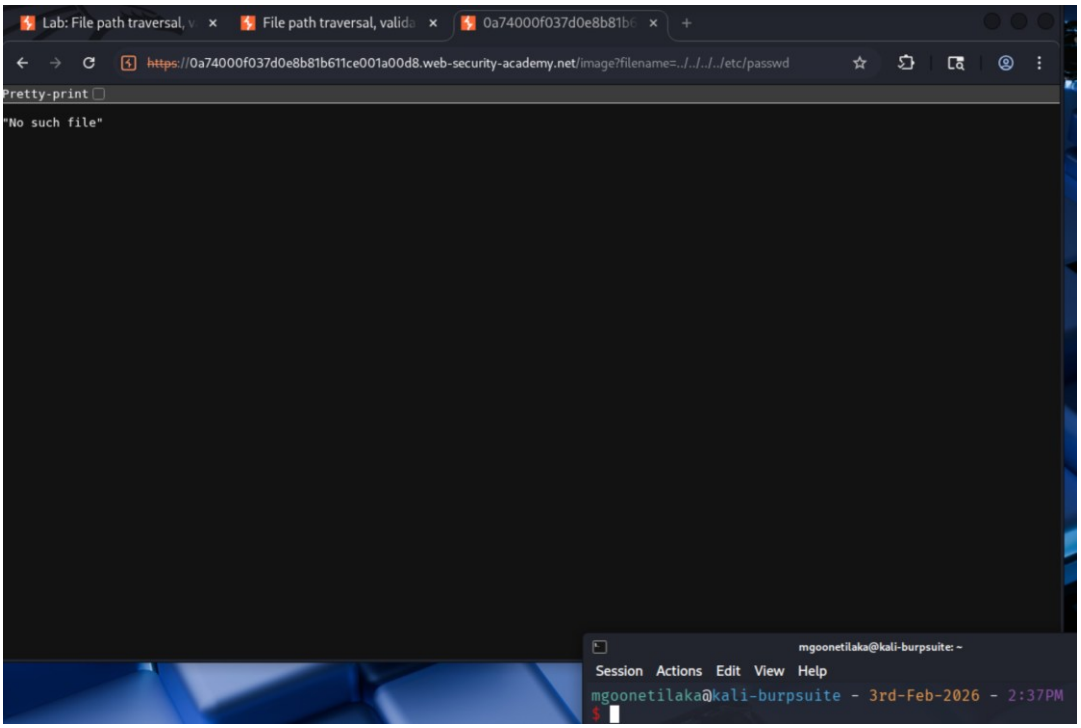

Figure 17: Standard traversal payload blocked by the application

This screenshot **(Figure 17)** shows the browser returning a "No such file" response when a standard ../../../../etc/passwd traversal payload was used in the filename parameter, indicating that the application requires the filename to end with a valid file extension such as .jpg.

2. The request was sent to Burp Suite Repeater, and the payload was modified to include a URL-encoded null byte (%00) followed by the .jpg extension. The final payload used was ../../../../etc/passwd%00.jpg. The null byte caused the server's filesystem API to terminate the path at /etc/passwd, while the .jpg suffix satisfied the application's extension validation check.

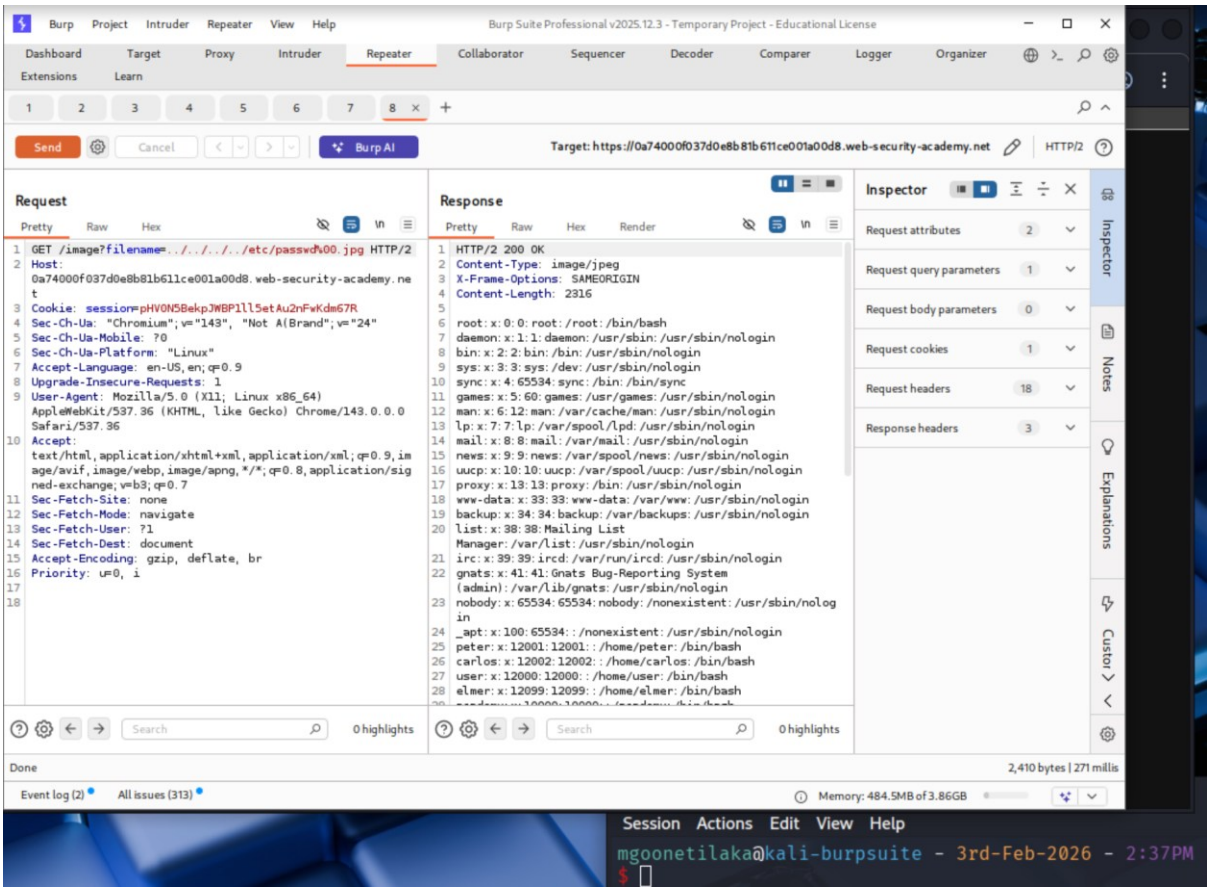

Figure 18: Null byte bypass payload in Burp Repeater

This screenshot **(Figure 18)** shows the Burp Suite Repeater with the modified GET request containing the payload ../../../../etc/passwd%00.jpg in the filename parameter. The server responded with HTTP/2 200 OK and returned the full contents of the /etc/passwd file in the response body, confirming that the null byte successfully terminated the file path before the .jpg extension was processed by the filesystem.

3. The lab was marked as solved after successfully retrieving the /etc/passwd file contents using the null byte bypass technique.

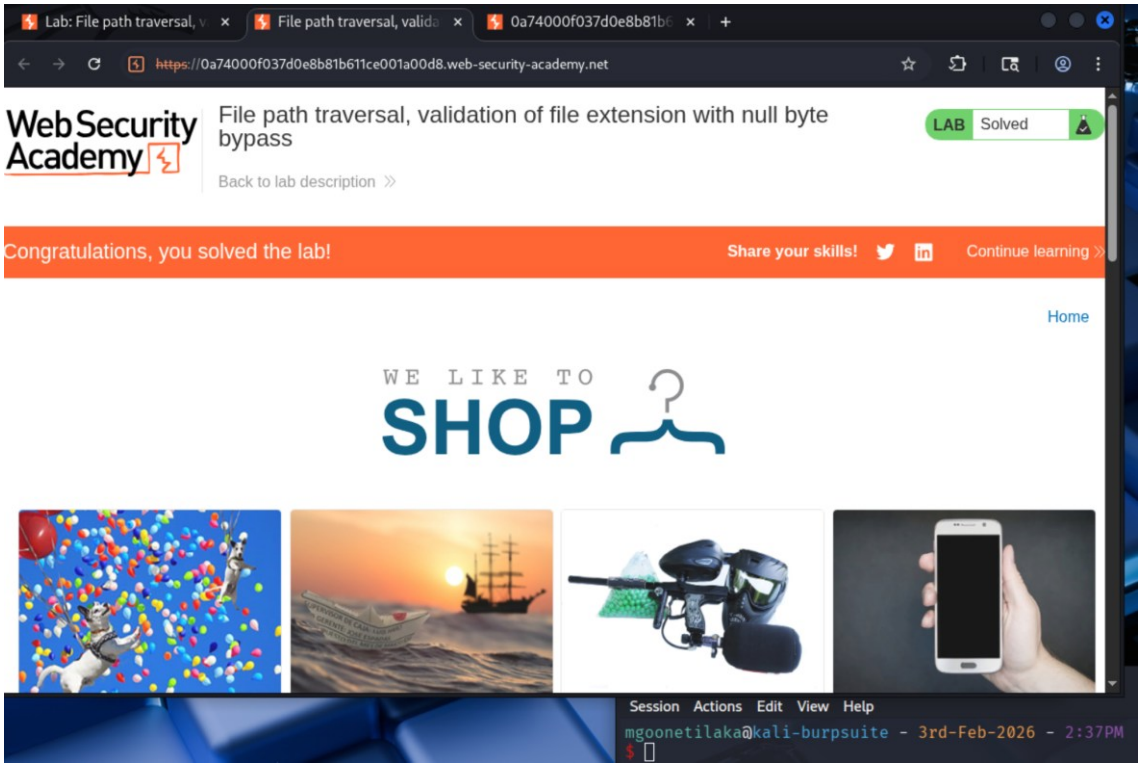

Figure 19: Lab solved confirmation

This screenshot **(Figure 19)** shows the application displaying the "Congratulations, you solved the lab!" confirmation message with the lab status marked as "Solved", indicating successful exploitation of the path traversal vulnerability using the null byte file extension bypass.

Lab Completion Status

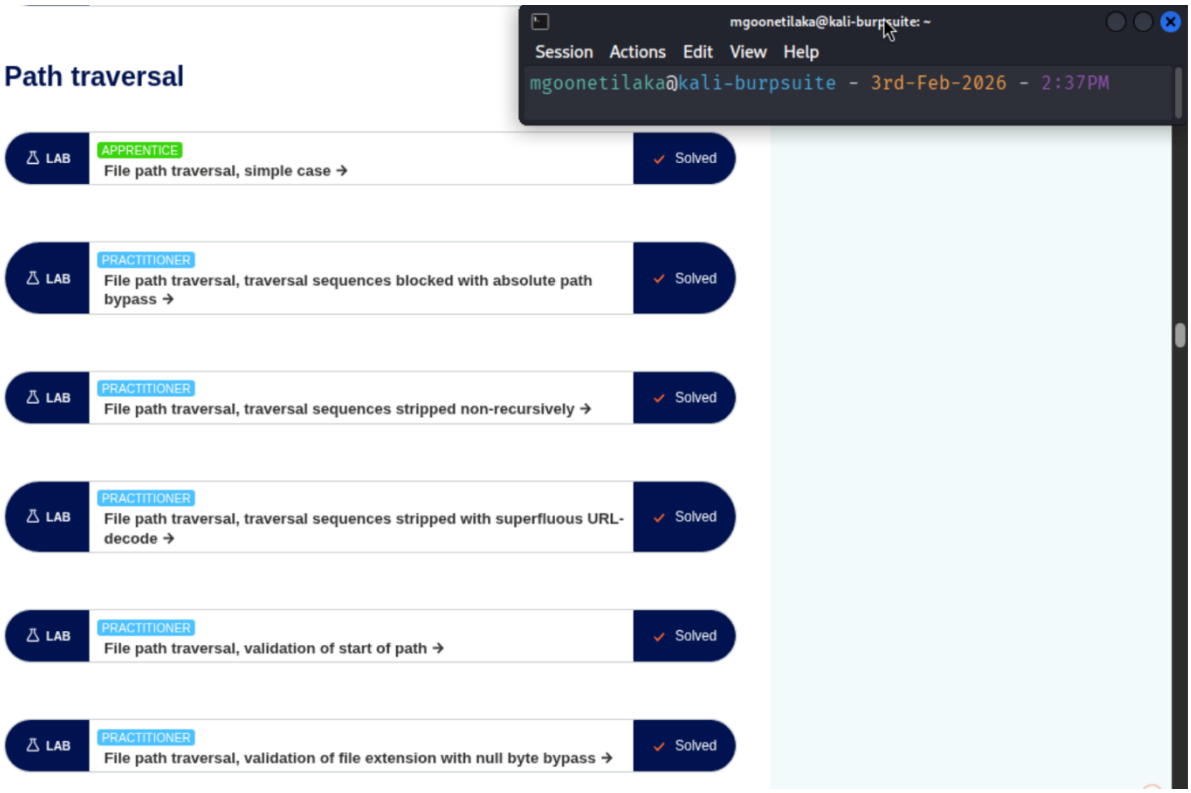

Figure 20: Lab Completion Status

Conclusion

This lab provided practical experience in identifying and exploiting path traversal vulnerabilities across different filtering and validation mechanisms. The six challenges showed that basic defenses like stripping traversal sequences or checking file extensions can often be bypassed through techniques such as nested payloads, encoding tricks, and null byte injection.

A key takeaway from these exercises is that relying on a single layer of input validation is not enough to prevent path traversal attacks. Effective defenses should include validating input after all decoding steps, canonicalizing file paths before use, and restricting file access to specific directories through whitelisting rather than blacklisting. These labs reinforced how common misconfigurations in file handling logic can lead to serious security issues in real-world web applications.

References

PortSwigger Web Security Academy. (n.d.). *File path traversal, simple case*. https://portswigger.net/web-security/file-path-traversal/lab-simple

PortSwigger Web Security Academy. (n.d.). *File path traversal, traversal sequences blocked with absolute path bypass*. https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass

PortSwigger Web Security Academy. (n.d.). *File path traversal, traversal sequences stripped non-recursively*. https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively

PortSwigger Web Security Academy. (n.d.). *File path traversal, traversal sequences stripped with superfluous URL-decode*. https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode

PortSwigger Web Security Academy. (n.d.). *File path traversal, validation of start of path*. https://portswigger.net/web-security/file-path-traversal/lab-validate-start-of-path

PortSwigger Web Security Academy. (n.d.). *File path traversal, validation of file extension with null byte bypass*. https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass